D1 / RAPPORT TECHNIQUE / 29 SEPTEMBRE 2026

Un agent vocal local, interruptible et mesurable

Python · Français · RTX 3070 Ti visée · Pipeline STT / LLM / TTS

1. Périmètre et état de la livraison

Le projet réalise un assistant vocal en terminal, avec capture continue, détection de fin de parole, génération de texte en flux, synthèse par phrase et lecture interruptible. Le code sépare la logique de conversation, les moteurs et les mesures pour rendre les erreurs et les décisions de concurrence testables.

Livré et vérifié ici	À mesurer sur le poste cible
52 tests réussis ; contrôle Ruff propre ; quatre essais CLI ; JSONL, CSV, statistiques et graphiques générés.	Qualité de transcription, latence réelle, interruption acoustique, annulation d'écho, VRAM et stabilité sur 30 minutes.
Sources modulaires, configurations, dépendances figées, scripts de corpus, protocole E1-E6 et scénario vidéo.	20 prises de voix humaine, 20 phrases Piper, 10 signaux d'interruption, bruit de pièce et vidéo live de deux minutes.

Les exécutions fournies utilisent des doublures déterministes, sans GPU ni périphérique audio. Elles valident la circulation des données et les calculs. Elles ne constituent pas une expérience de performance sur la RTX. Aucun résultat réel E1-E6 n'est inventé.

L'objectif reste p50 ≤ 1,2 s et p95 ≤ 2,0 s entre la dernière trame de parole et le premier audio. Le coût d'API de cette livraison est nul. La comparaison Realtime et le routage automatique sont des extensions P2 non implémentées.

Pièces de preuve : docs/test-results.txt ; runs/validation-simulated ; runs/validation-replay ; runs/validation-interruptions ; runs/validation-text.

2. Architecture et interruption

Un orchestrateur asyncio possède chaque tour et ses tâches. Ollama reste un serveur local séparé. Les moteurs natifs tournent hors de la boucle événementielle.

Étage	Implémentation	Décision de conception
Entrée / VAD	PortAudio, 16 kHz, 512 échantillons ; Silero ONNX CPU	File bornée, pré-roll, seuil normal 0,5 et seuil d'interruption 0,7.
Transcription	faster-whisper, français, int8_float16	Segment terminé, un exécuteur dédié ; repli small/CPU configurable.
Réponse	Ollama Qwen2.5 3B Q4_K_M	NDJSON, historique de six tours, file de quatre phrases, segmentation bornée.
Voix / lecture	Kokoro Siwis, ou Piper CPU ; sortie 24 kHz	File de trois unités audio, callbacks de 20 ms, estimation de l'heure DAC.
Traces	Bus unique, perf_counter_ns, JSONL	Écriture hors boucle, configuration et versions enregistrées avec la session.

Interruption. La parole candidate doit durer dix trames, soit 320 ms. La lecture est invalidée et abortée en premier ; les tâches LLM/TTS/lecture sont ensuite annulées. Le flux HTTP est fermé. Les trames déjà enregistrées servent au nouveau tour, sans perdre le début de la demande.

Annulation native. Une inférence CUDA déjà lancée ne peut pas être tuée par Task.cancel(). Son résultat est abandonné. Un exécuteur à un seul thread par moteur empêche une nouvelle inférence du même moteur de chevaucher l'ancienne. Le prochain tour peut attendre ce calcul restant.

Historique. Seules les unités TTS entièrement jouées sont conservées. Une phrase coupée est omise et le tour porte [interrompu]. Cette méthode ne prétend pas connaître l'alignement exact entre mots français et échantillons.

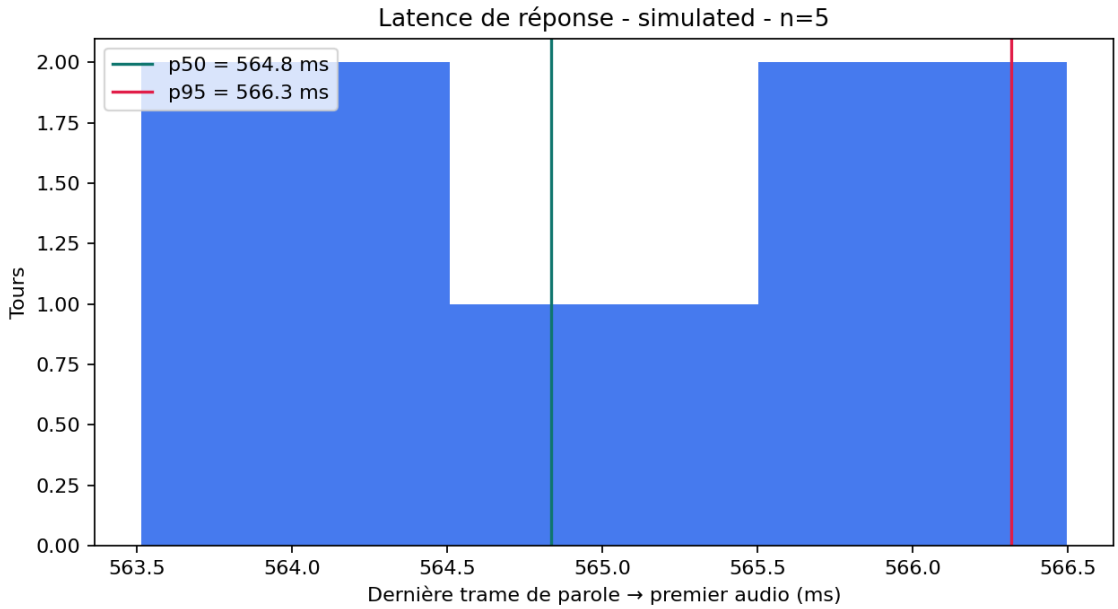
Écho. Le module PipeWire WebRTC doit être routé vers le micro et les haut-parleurs. Une porte d'énergie compare le micro à une estimation de l'écho (-10 dB par défaut, marge 6 dB). Ce réglage doit être calibré. Le mode half-duplex est disponible, avec interruption désactivée.

3. Mesures de validation logicielle

Les valeurs ci-dessous proviennent de la session de simulation incluse. Les moteurs renvoient des textes fixes et des sinusoïdes. Ces mesures vérifient les horodatages et le rapport ; elles ne prouvent pas l'atteinte des objectifs matériels.

Étape	n	p50 simulé	p95 simulé
Fin de tour	5	512.0 ms	512.0 ms
STT simulé	5	12.2 ms	13.7 ms
Premier fragment LLM	5	3.2 ms	3.3 ms
Première phrase	7	31.7 ms	32.4 ms
TTS simulé	7	5.4 ms	5.8 ms
Démarrage simulé	7	0.1 ms	0.1 ms
Réponse, bout en bout	5	564.8 ms	566.3 ms

Réponse : n = 5, IC95 du p50 par bootstrap [563.5 ; 566.5] ms. Avec moins de 20 tours, le p95 est peu fiable. Le nombre de phrases mesurées peut dépasser celui des tours vocaux, car la session contient aussi du texte.



Le budget nominal du PRD additionne 500 + 250 + 200 + 250 + 150 + 30 = 1 380 ms. Il dépasse déjà la cible de 1 200 ms. Il faudra donc réduire un ou plusieurs postes. Les médianes par étage ne s'additionnent pas nécessairement à la médiane totale.

4. Protocole expérimental et résultats disponibles

Expérience	Méthode fournie	État réel
E1 - latence	40 WAV, même VAD et pipeline, warmup exclu, horloge monotone.	Non mesurée sur cible.
E2 - WER	Blanc, rose, bruit réel ; propre, 20, 10, 5, 0 dB ; référence FR.	Calcul testé ; corpus à enregistrer.
E3 - interruption	Injection à 2 s, écho -10 dB / 30 ms, réponse fixe.	Un cas technique simulé accepté.
E4 - faux départ	Toux, euh et bruits courts annotés.	Un cas technique simulé rejeté.
E5 - AEC	20 réponses avec AEC, puis 20 sans ; même volume et placement.	À réaliser dans la pièce.
E6 - live	20 interruptions humaines, annotation et contrôle acoustique.	À réaliser sur le PC.

Statistiques. Le rapport calcule n, p50/p95, min/max pour les petits échantillons, IC95 du p50 par bootstrap (1 000 tirages) et intervalles de Wilson pour les proportions. Un WER est agrégé par nombre de mots, pas par moyenne des pourcentages des phrases. Les cas non transcrits comptent comme des suppressions.

Interprétation des fixtures. Le cas E3 simulé donne 1/1 accepté, mais son intervalle de Wilson à 95 % est environ [20,7 % ; 100 %]. E4 donne 0/1 faux départ, avec borne haute proche de 79,3 %. Ces minuscules effectifs démontrent le chemin de calcul, sans étayer une performance de 90 %.

Conditions distinctes. Il y a 40 × (1 + 4 × 3) = 520 tours distincts. Les 600 tours du produit brut répètent le propre trois fois. Le script permet cette grille complète, mais ces répétitions ne sont pas indépendantes.

Analyse d'erreurs à mener. Examiner séparément les endpoints manqués, les hallucinations sur bruit, les mots rares, les coupures trop précoces et les auto-interruptions par écho. Annoter la vérité terrain avant de calculer un taux. Ne jamais déduire une fausse interruption de la seule sortie du VAD.

VRAM cible : ≤ 7 Go, non mesurée ici. nvidia-smi couvre la carte entière à 1 Hz ; le pic PyTorch ne couvre pas Ollama ni CTranslate2. Les pics brefs et l'arrêt acoustique exigent des observations complémentaires.

5. Reproduction, limites et validation finale

L'installation complète et les replis sont détaillés dans README.md et docs/INSTALLATION.md. Python 3.12 est isolé dans un environnement uv. Le fichier de dépendances complet a été résolu ; l'installation CUDA doit être confirmée sur la machine cible.

Ordre	Action
1	Installer uv, Ollama, PortAudio et espeak-ng ; vérifier nvidia-smi.
2	Créer l'environnement Python 3.12 et installer requirements-local.lock.txt.
3	Démarrer Ollama et télécharger qwen2.5:3b-instruct-q4_K_M.
4	Lancer prepare : snapshots Hugging Face et digest Ollama figés.
5	Essayer run --text, puis configurer PipeWire et passer au mode vocal.

Avant le rendu. Faire cinq tours vocaux sans clavier ; vérifier que la lecture précède la fin du LLM ; tester l'interruption et un bruit court ; mesurer la VRAM ; produire le corpus ; lancer E1-E6 ; générer les rapports de ces sessions. La vidéo de deux minutes doit montrer le poste réel. Le storyboard est fourni, mais aucune vidéo matérielle ne peut être certifiée depuis ce conteneur.

Limites qui restent visibles. STT par segment et non incrémental ; mémoire à la phrase ; validation d'interruption à 320 ms minimum ; arrêt hôte distinct de l'arrêt du son dans la pièce ; porte d'énergie non calibrée ; objectifs de vitesse et de robustesse non validés. La route cloud P2 et le routage automatique sont explicitement absents. Les sources sont prêtes pour GitHub, sans publication externe.

Critère de réussite final. Les tests logiciels sont nécessaires mais insuffisants. Le dossier devient une preuve expérimentale lorsqu'il contient les WAV autorisés, les traces réelles, les annotations, les chiffres calculés et une démo reproduite sur le matériel annoncé.

Références techniques

faster-whisper : github.com/SYSTRAN/faster-whisper
Kokoro : github.com/hexgrad/kokoro
Silero : github.com/snakers4/silero-vad
Ollama : docs.ollama.com/api/chat
Audio : python-sounddevice.readthedocs.io/en/latest/api/streams.html
AEC : docs.pipewire.org/page_module_echo_cancel.html
Piper : github.com/OHF-Voice/piper1-gpl

Consultées le 29 septembre 2026. Liens complets et interfaces inspectées dans docs/SOURCES.md. Cahier des charges : document fourni avec la demande.